INF1103: Lab 5

Data Manipulation

1. Introduction

Welcome to Week 5! In previous weeks, we saved our store data to a text file.

Remember, the focus of this module is all about bridging logical programming constructs with modern, automated operational practices. We will systematically evolve our applications week by week, refactoring and packaging our code to fit enterprise-grade deployment standards.

The expectation is that you complete and verify your lab by the end of your scheduled lab session. True engineering mastery comes from hands-on problem-solving. Ensure you work through and complete each step of this lab by yourself. While your peers are excellent resources for discussing conceptual architectures, they must not write code for you or share their files with you!

It is highly critical that you have watched and engaged with the Week 1 lecture lessons before attempting this lab. We assume you have acquired the foundational mental models—like standard shipping containers, blueprints vs. instances, and environment drift—which you will now put into practice.

Do not hesitate to ask plenty of questions to your professor and teaching assistants when you hit an infrastructure error. We are here to support you as you take your first steps into the DevOps pipeline. Good luck!

Text files are simple and useful for small programs, but they lack structure, are difficult to update and search, and do not scale well for large amounts of data, which is why we use JSON and databases.

Remember:

- Data manipulation is the process of creating, reading, updating, and deleting (CRUD) data.
- As data grows larger, databases become more efficient than files for storing, searching, and managing information.

2. Assignment

1. The Scenario

The store manager needs a system that remembers inventory levels even after the program closes. Furthermore, they need to store a history of all transaction amounts, not just the running total.

Requirements

1. **Data Representation:**
   Represent inventory items using dictionaries and store at least three products in a list.
2. **Data Manipulation:**
   Maintain your functional design. Create add_product(), update_stock(), search_product() and display_all() fuctions in inventory dictionary.
3. **Data Persistence:**
   Check whether inventory.json exists. Create load_inventory() to load inventory.json if it exists. Otherwise, begin with an empty inventory. Create save_inventory() and save data to inventory.json.
4. **Build a Menu System:**
   Create menu options for Display, Add, Update, Search, Save and Exit.

Sample Output:

```
==========================================
INVENTORY MANAGEMENT SYSTEM
==========================================

inventory.json found.
Inventory loaded successfully.

---------- MENU ----------
1. Display All Products
2. Add Product
3. Update Stock
4. Search Product
5. Save Inventory
6. Exit
--------------------------

Enter option: 1

Current Inventory
------------------------------------------------
ID: P001 | Name: Laptop | Price: $1200.00 | Stock: 15
ID: P002 | Name: Mouse | Price: $25.50 | Stock: 40
ID: P003 | Name: Keyboard | Price: $45.00 | Stock: 25
------------------------------------------------

Enter option: 2

Add New Product
Product ID: P004
Product Name: Monitor
Price: 299.99
Stock Quantity: 10

Product added successfully!

Enter option: 1

Current Inventory
------------------------------------------------
ID: P001 | Name: Laptop | Price: $1200.00 | Stock: 15
ID: P002 | Name: Mouse | Price: $25.50 | Stock: 40
ID: P003 | Name: Keyboard | Price: $45.00 | Stock: 25
ID: P004 | Name: Monitor | Price: $299.99 | Stock: 10
------------------------------------------------
```

```
Enter option: 3

Update Stock
Enter Product ID: P002

Product Found:
Name: Mouse
Current Stock: 40

New Stock Quantity: 50

Stock updated successfully!

Enter option: 4

Search Product
Enter Product ID: P004

Product Found
------------------------------------------------
ID: P004
Name: Monitor
Price: $299.99
Stock: 10
------------------------------------------------

Enter option: 5

Saving inventory...
Inventory saved successfully to inventory.json.

Enter option: 4

Search Product
Enter Product ID: P999

Product not found.

Enter option: 6

Saving inventory before exit...
Inventory saved successfully.

Thank you for using Inventory Management System.
Program terminated.
```

2. Initialize Your Workspace & Git

1. Initialize and track your work.
2. Commit your progress after you successfully complete each phase:
    a. After creating the Dictionary and adding few products
    b. After load_inventory() is working.
    c. After the final save_inventory() is verified.
3. Finally push the code to your github repository.

3. Dockerize with Volumes

To ensure your file persists outside the container even if the container is destroyed, run it with a volume mount.

What you need to submit?

1. Your python file (inventory_manager.py)
2. Your updated inventory.json file
3. Your git log. Store it in a text file.
   git log >log.txt
4. Docker Mount and run. A screenshot of the successful execution is usually sufficient.
5. Run the following command for evidence that the image was successfully built. Screenshot of the output.

   ```
   docker images
   ```

6. Proof that the image runs successfully with a screenshot showing the program accepting input and producing results.

7. Put everything in a folder. Zip it and upload to xSite.